

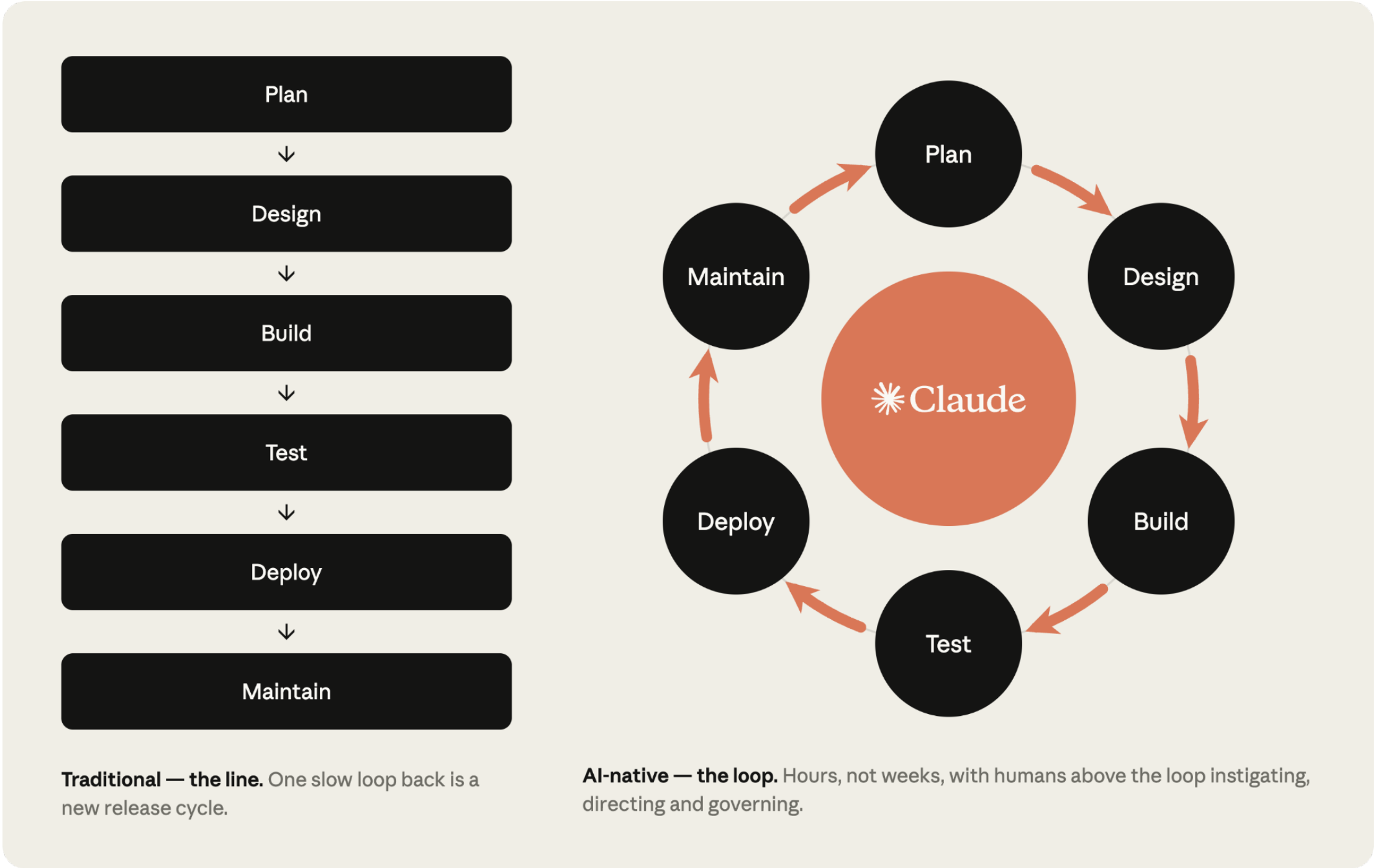
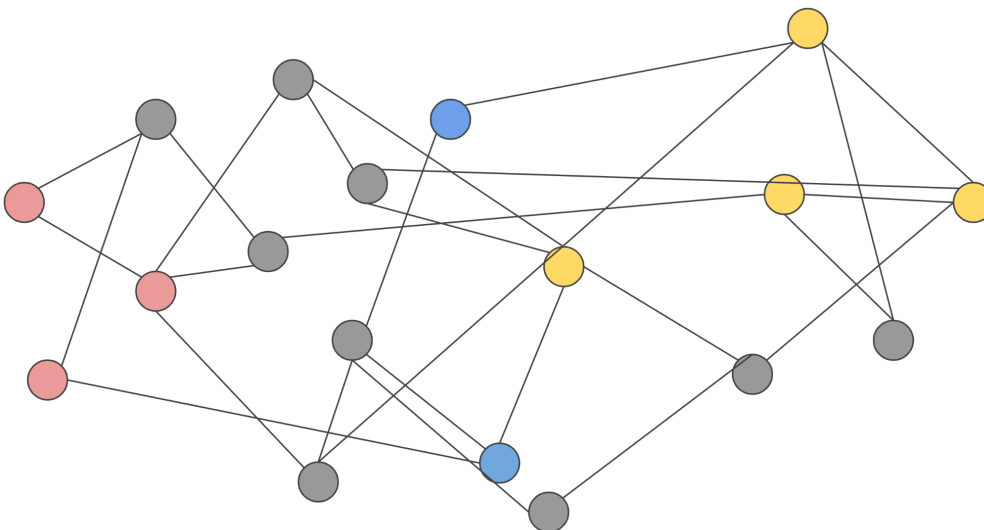
Partner with AI to Write Quality Code



Ethereum & AI Engineering Group

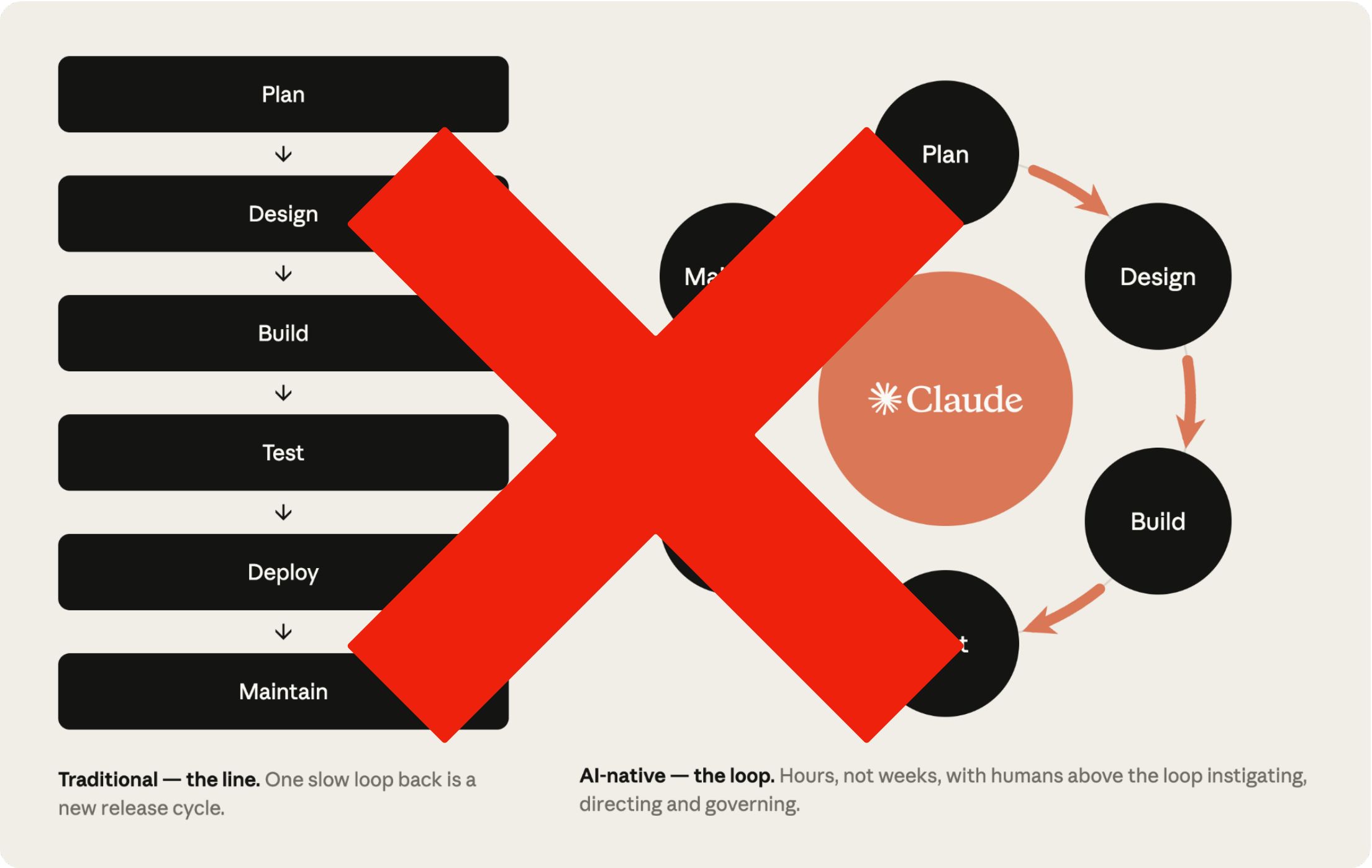
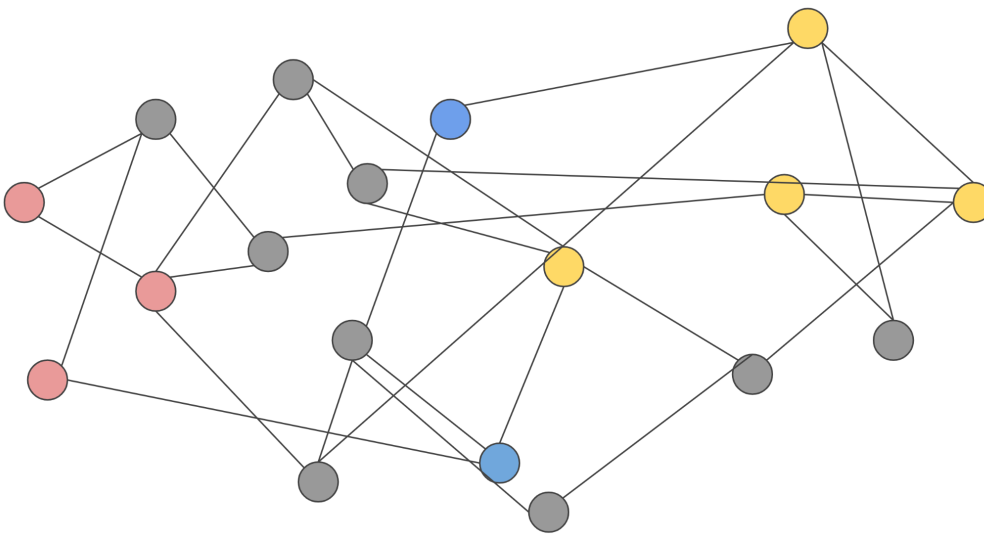
David Hyland-Wood - 30 September 2026

The Software Development Lifecycle

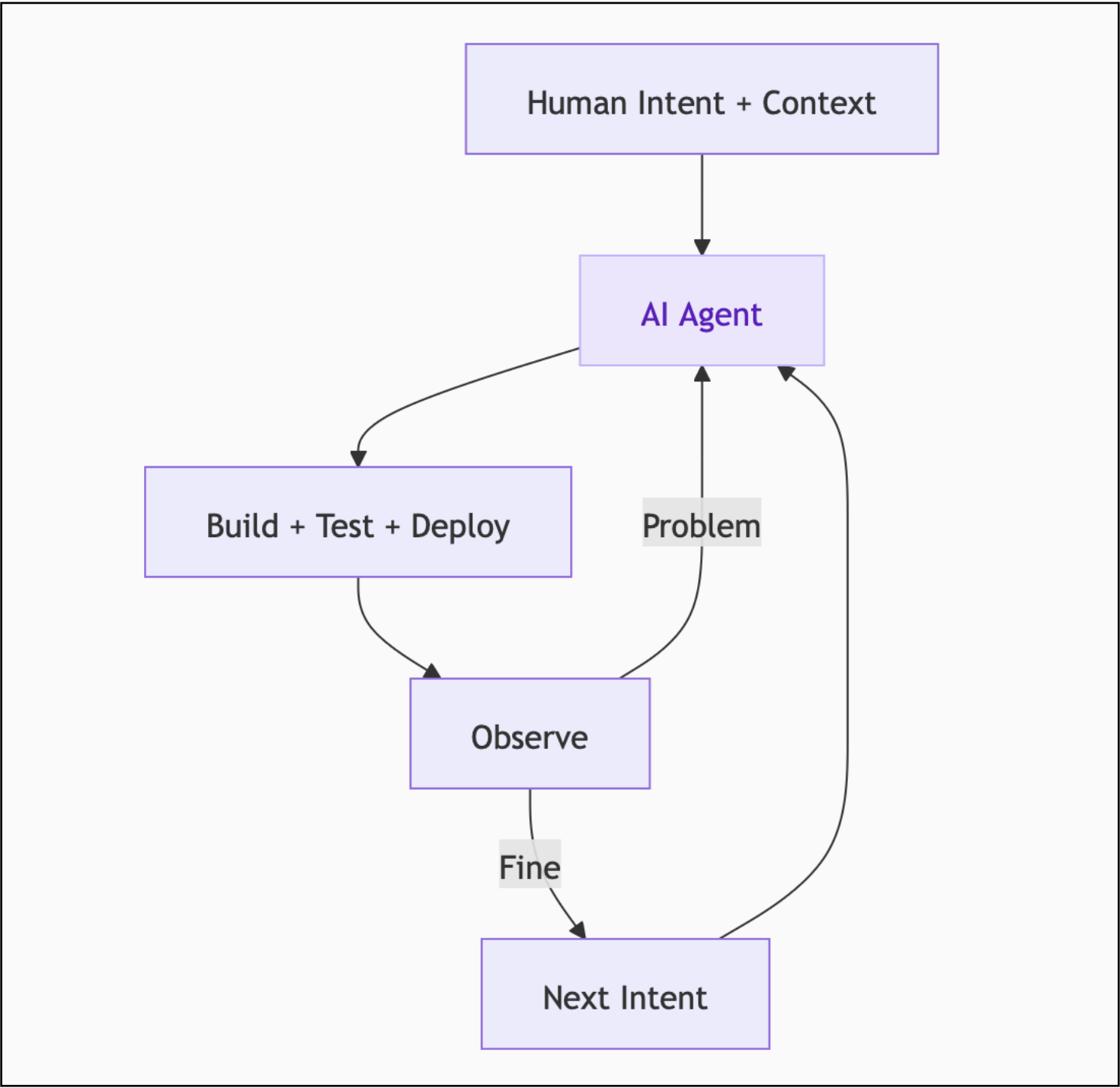


Source: Louis Claxton, The AI-Native SDLC playbook,
<https://claude.com/blog/the-ai-native-sdlc-playbook>,
21 August 2026

The Software Development Lifecycle

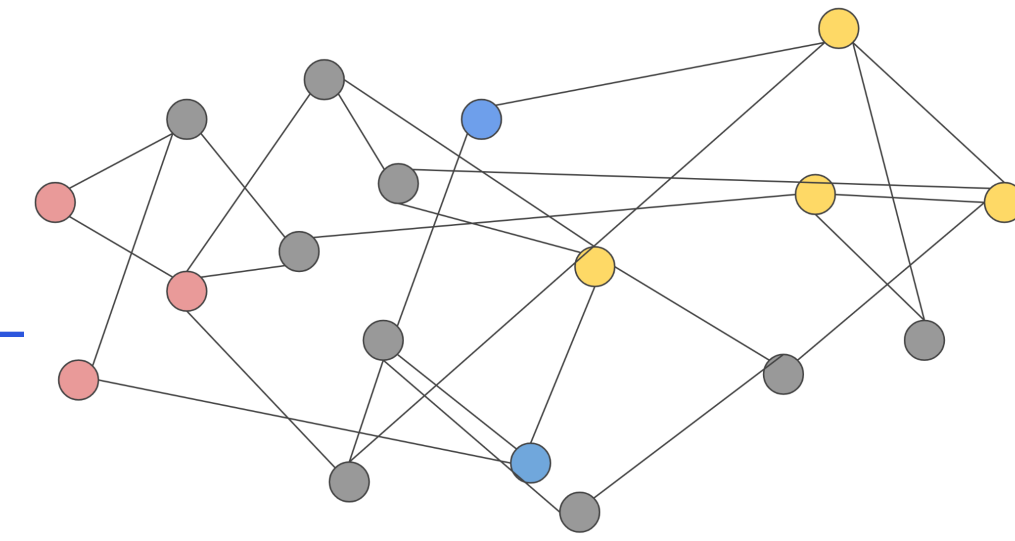


Source: Louis Claxton, The AI-Native SDLC playbook,
<https://claude.com/blog/the-ai-native-sdlc-playbook>,
21 August 2026



Source: Boris Tane, The Software Development Lifecycle Is Dead,
<https://boristane.com/blog/the-software-development-lifecycle-is-dead/>,
20 February 2026

Human Understanding of Code

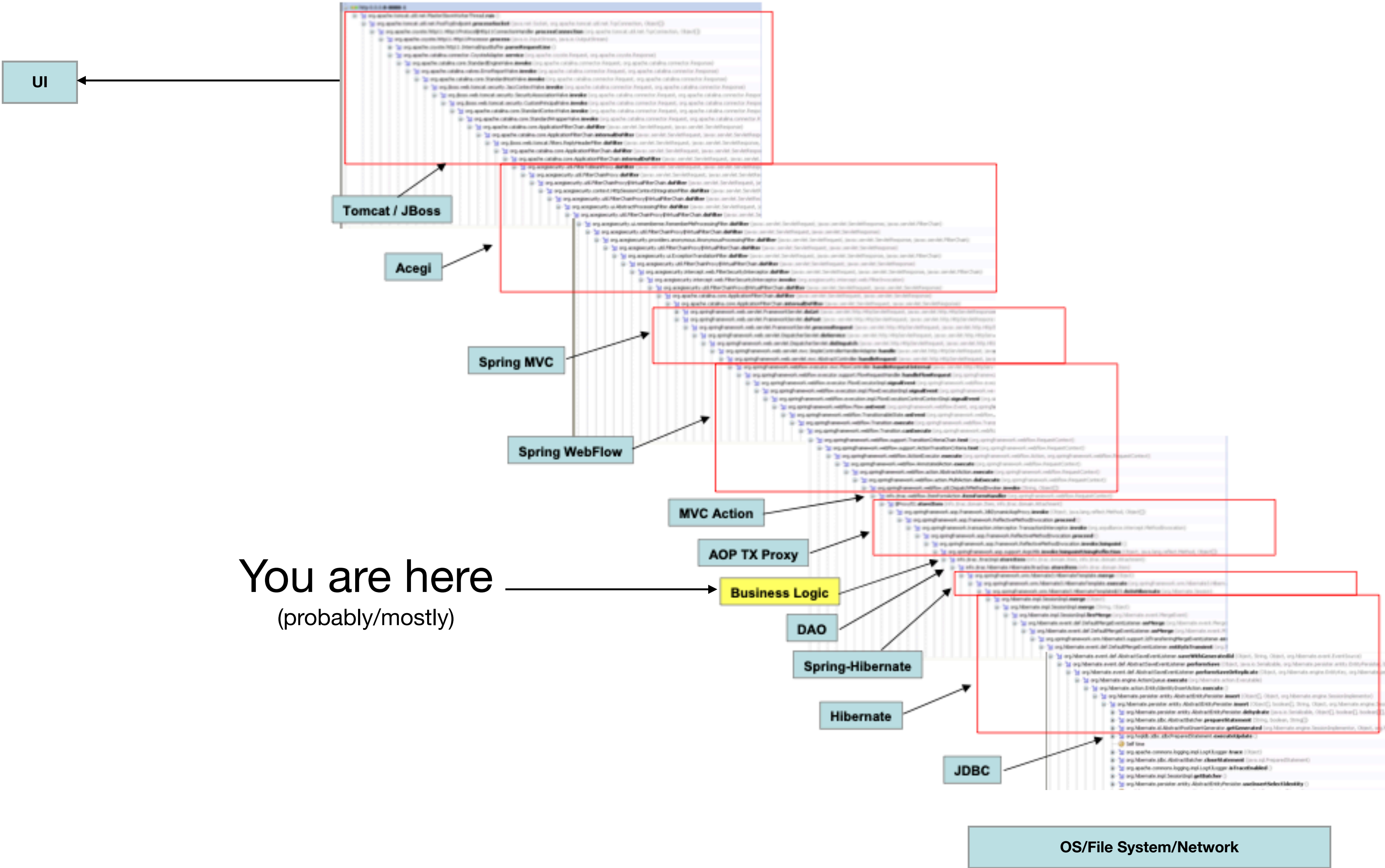
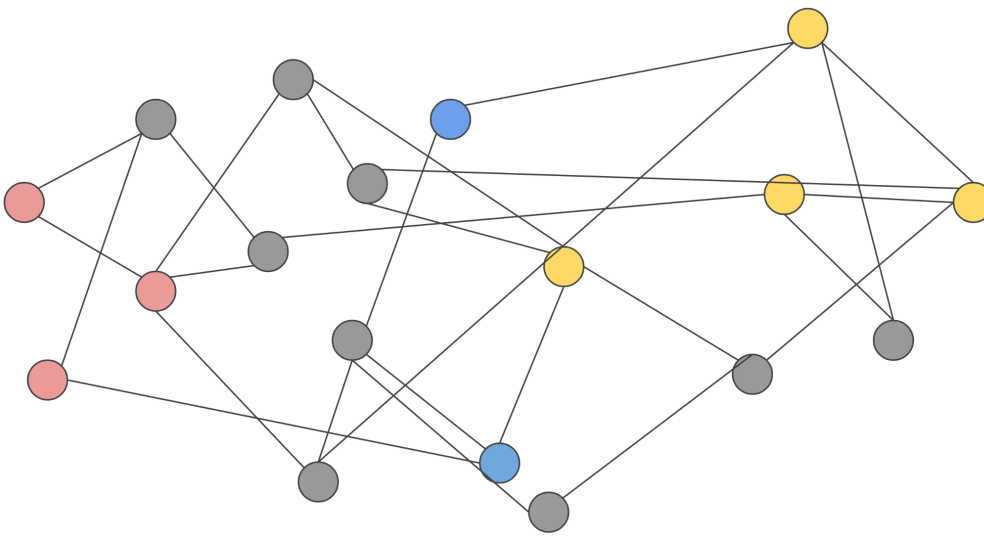


“I straight up asked them if they **know what their code does**. They didn’t. The PR was not approved.”

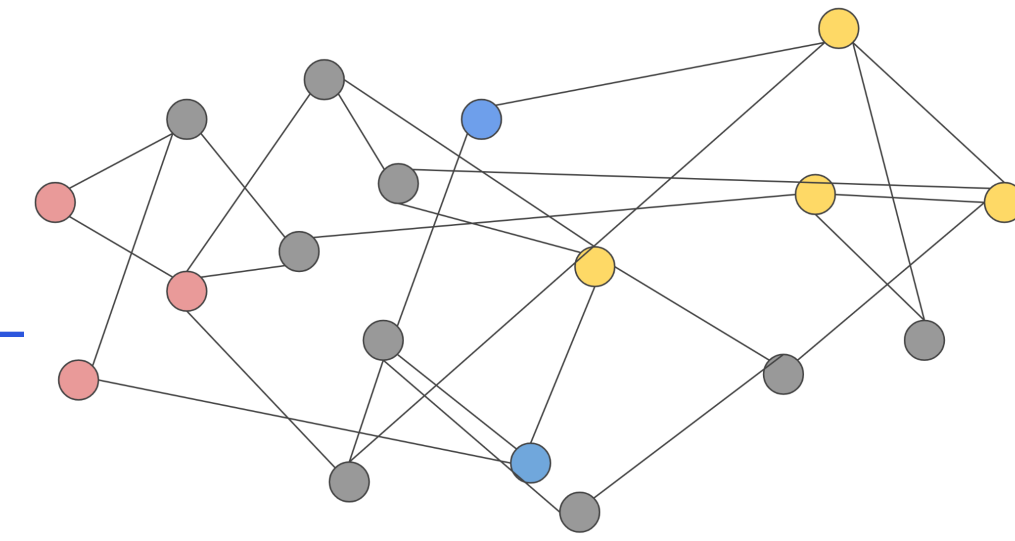
“**AI slop** is ripping up the social contract between maintainers and contributors essential to open source development”

Sebastian Baltes, Marc Cheong, and Christoph Treude,
"An Endless Stream of AI Slop': How Developers
Discuss the Burden of AI-Assisted Software
Development," preprint, arXiv, March 28, 2026, <https://doi.org/10.48550/arXiv.2603.27249>.

Really? How Abstracted are You??



Code Quality

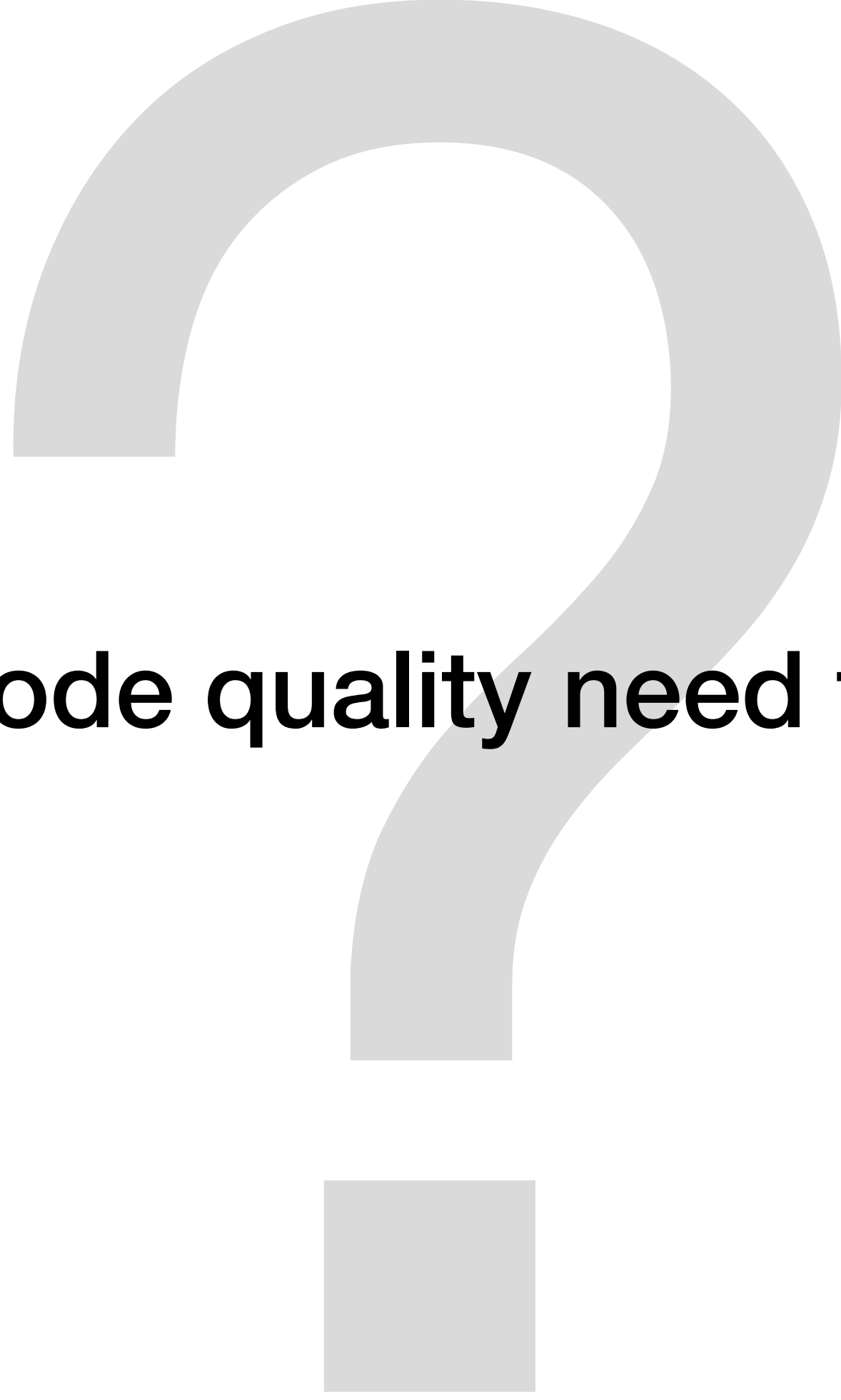


“Almost all code ever written is **mediocre (or worse)** and that’s what LLMs have been trained to replicate without understanding”

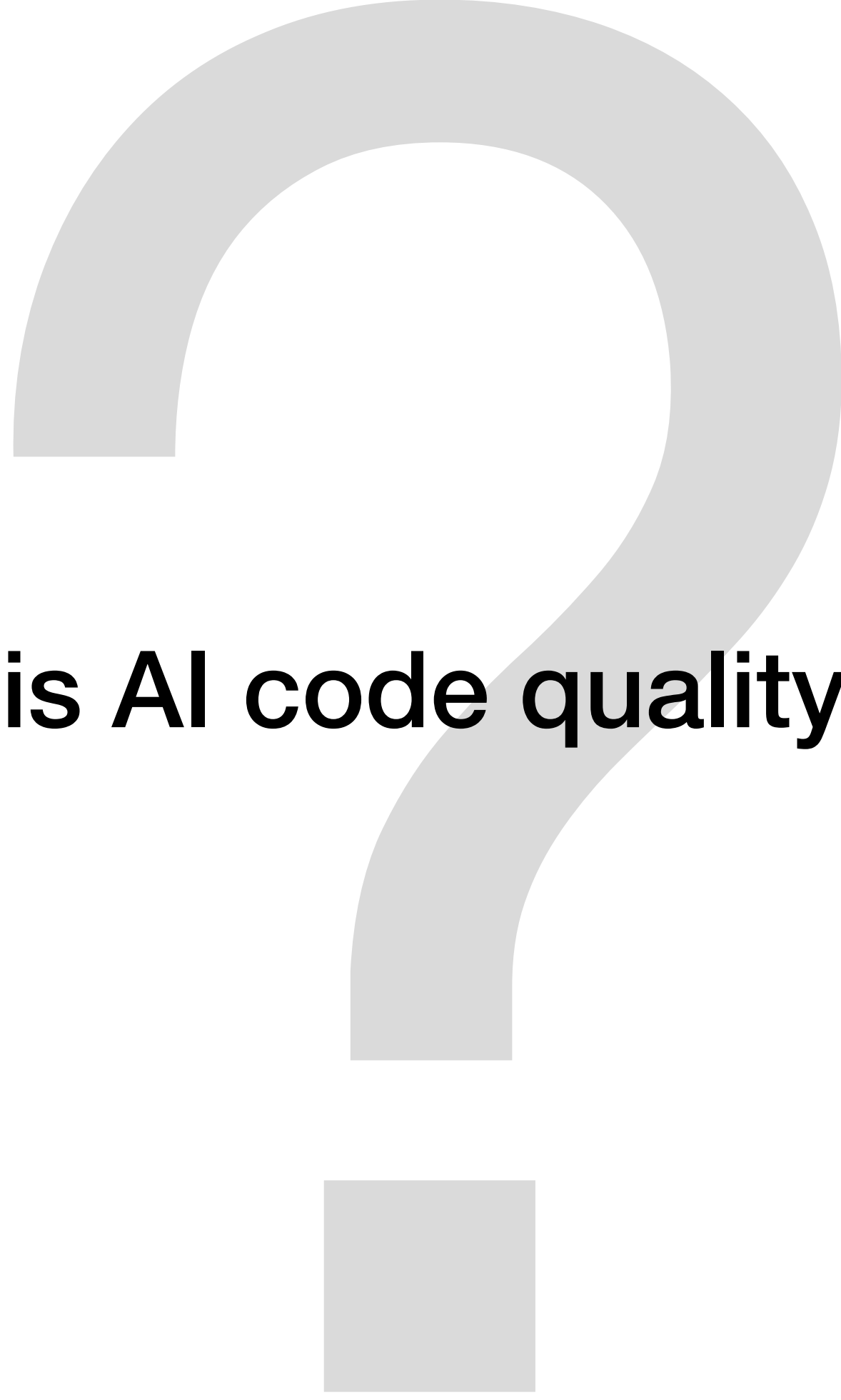
“What does saving an hour or two writing code, which I actually like, give me if I have to spend that same hour or two **deshittifying it?**”

“[There has been] a huge push for teams to adopt tools like Cursor [...] rapidly becoming an endless stream of **AI slop.**”

Sebastian Baltes, Marc Cheong, and Christoph Treude, "An Endless Stream of AI Slop': How Developers Discuss the Burden of AI-Assisted Software Development," preprint, arXiv, March 28, 2026, <https://doi.org/10.48550/arXiv.2603.27249>.

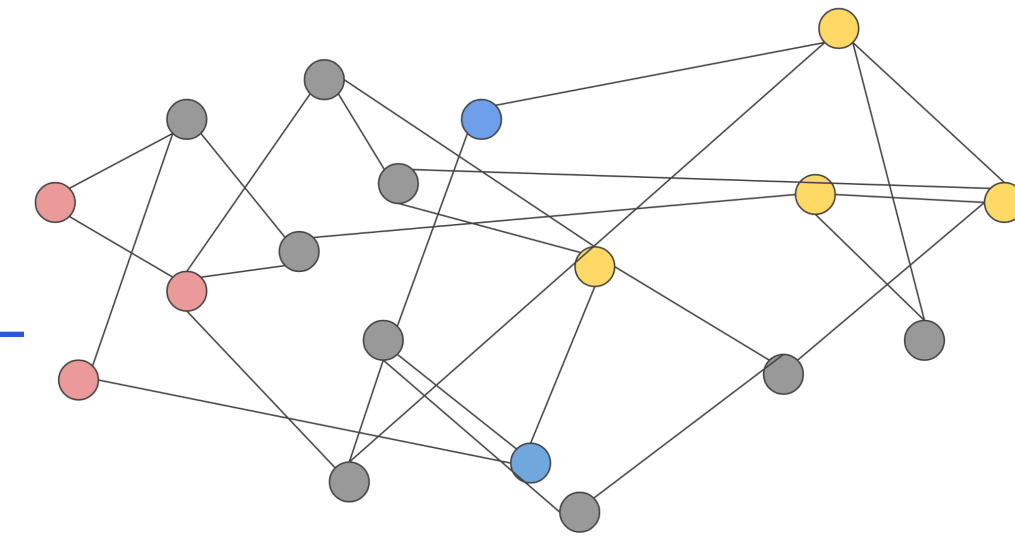


Does AI code quality need to be bad?



Why is AI code quality bad?

Zero Hallucination Is Mathematically Impossible



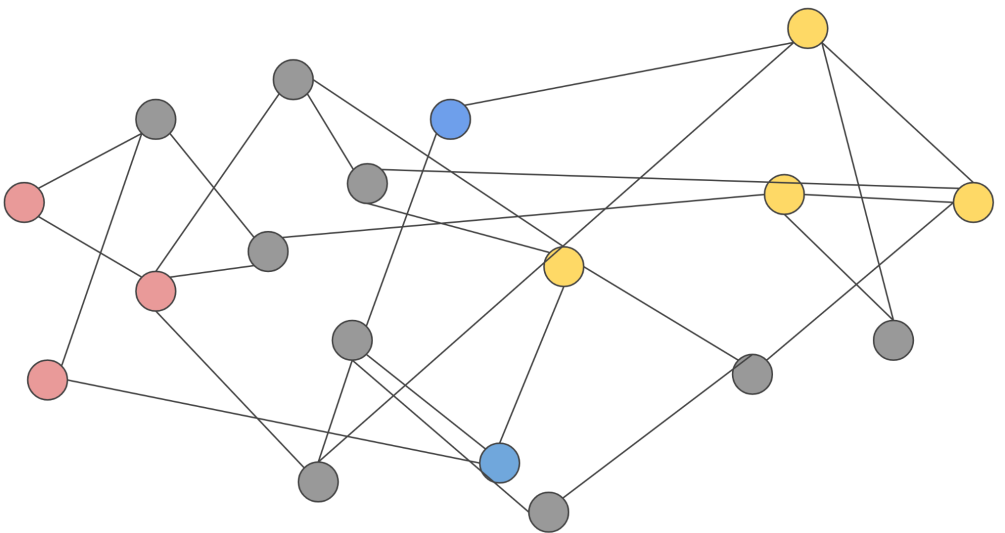
“We prove that perfect hallucination control in large language models is mathematically impossible.

No LLM inference mechanism can simultaneously achieve truthful response generation, semantic information conservation, relevant knowledge revelation, and knowledge-constrained optimality.

This impossibility is fundamental, arising from the mathematical structure of information aggregation itself rather than engineering limitations.”

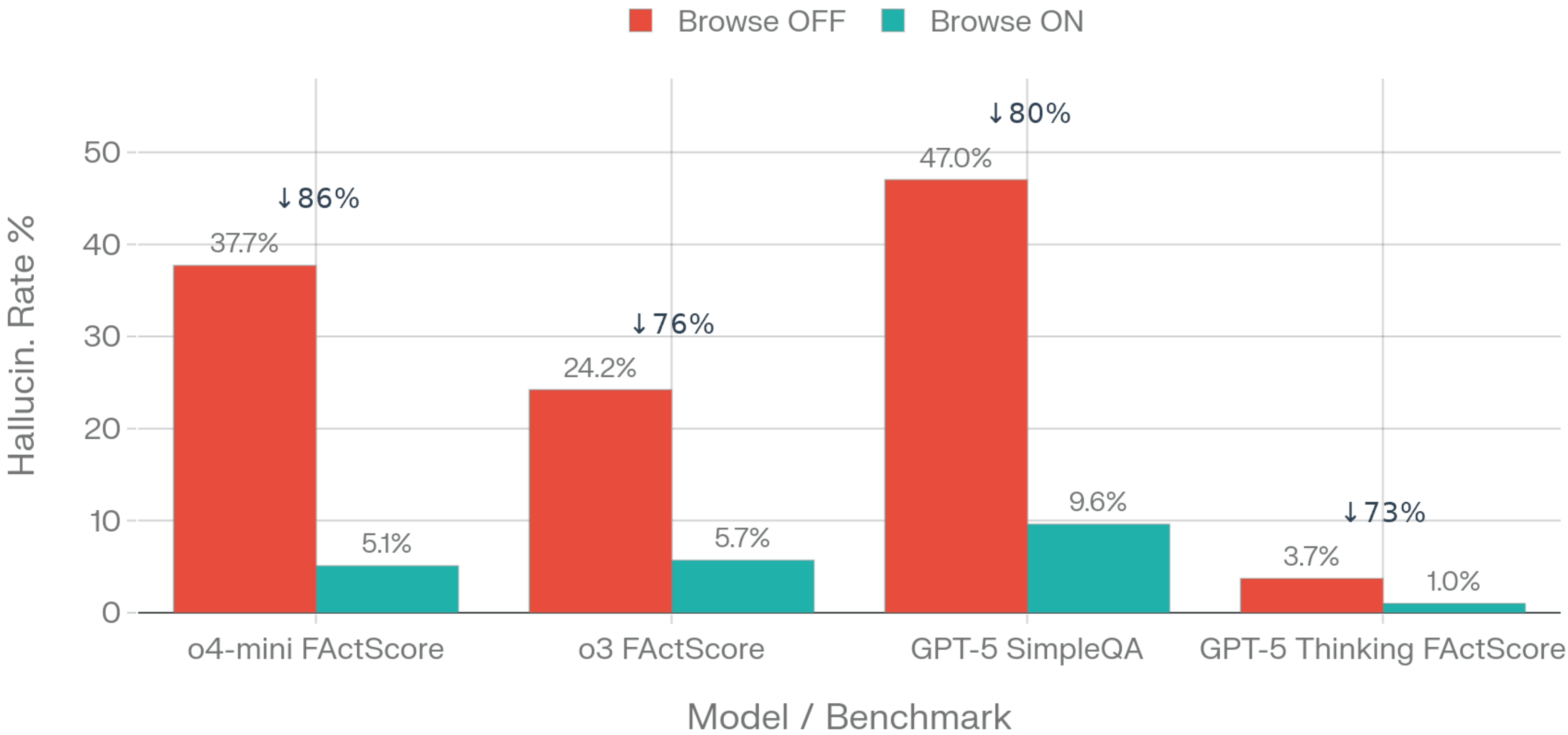
Michał P. Karpowicz, "On the Fundamental Impossibility of Hallucination Control in Large Language Models," preprint, arXiv, June 4, 2025, <https://arxiv.org/abs/2506.06382v3>.

Hallucinations

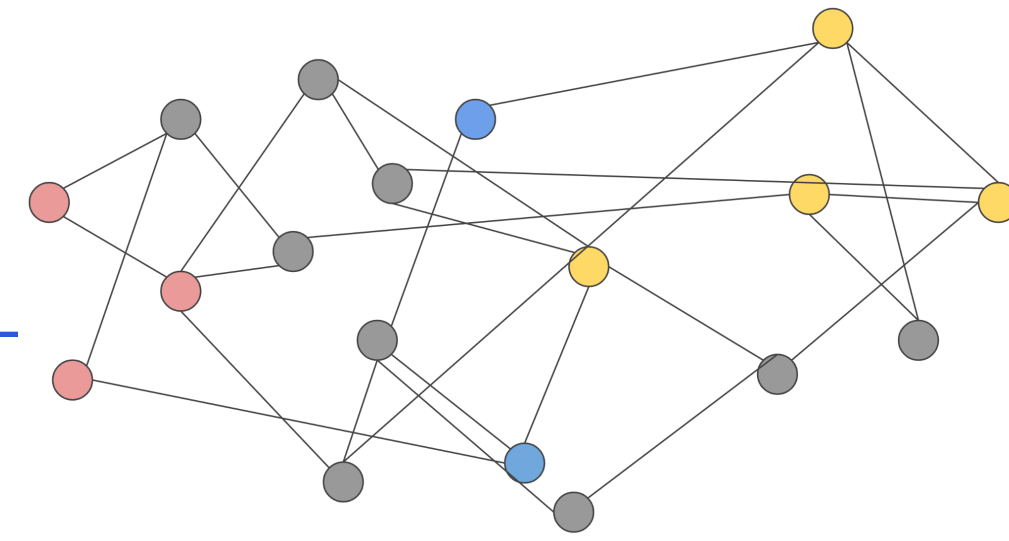


The Browse Effect: Web Access vs No Access

Source: OpenAI System Cards [8][9][11] | Browse-on reduces hallucination 73–86%



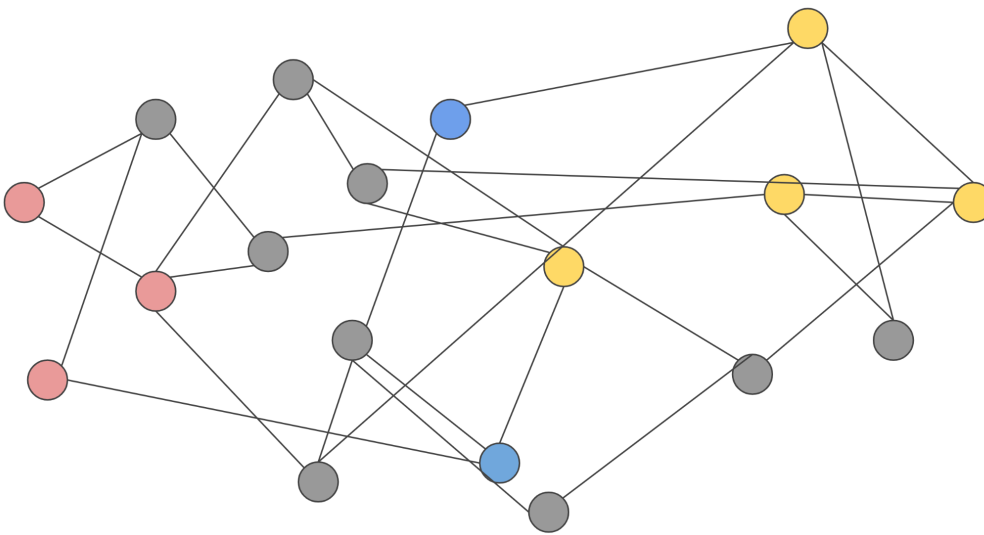
So how can AIs create good code?



“The right question isn't "which AI doesn't hallucinate?"
— **every AI hallucinates.**

The right question is: what systems do you have in place
to **catch hallucinations** before they reach a decision-
maker?”

Catching and Resolving Hallucinations



Claude Opus 5.5
(the model itself)

One nit I already spotted: the help text now says "type a number in the Order box" twice. I will get that tightened.

security-reviewer

The security gate earned its money again

It failed three times. Every failure was a real defect. Two were **fail-open holes inside guards that claimed to fail closed**.

Mutation testing found worse. W4's certificate bytes had **zero** tests behind them – five deliberate breakages all went unnoticed. Now fixed and pinned.

One finding stands out. A test looked honest but proved nothing, because the test data was too simple: for a root, three different identifiers all collapse to the same value. So three wrong readings of the spec all passed. Fixed by adding data where the three values differ.

PM report and cycles

- **Review done. Two reviewers, working apart.**

Good news: the important stuff is right. All page links still work. Nothing broke. The tests pass.

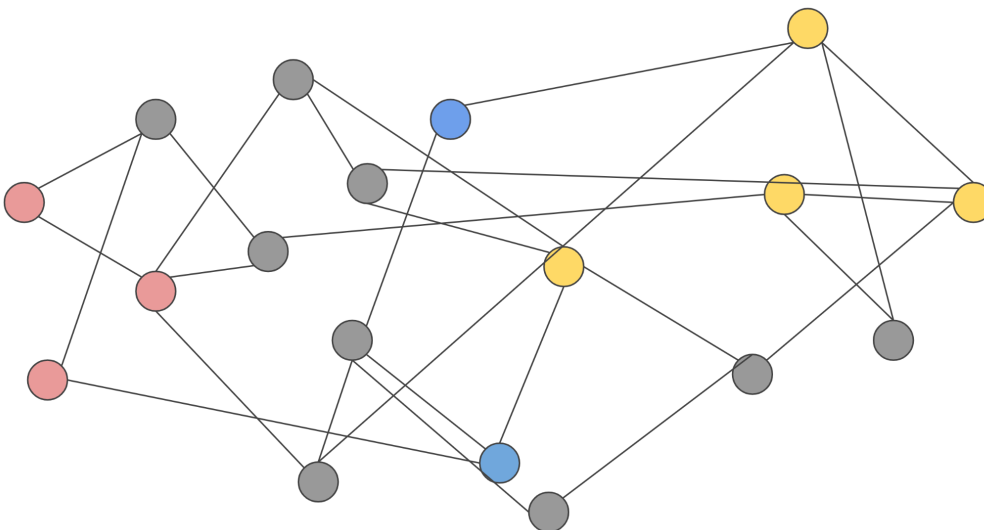
Six things to fix:

1. Single-source story link lists show at the wrong text size. Real bug you would see.
2. Some old CSS now does nothing. Delete it.
3. Trend line can end with "· updated " and no date.
4. It says "1 trends tracked". Bad grammar.
5. Ten lines of copy-pasted code. It will drift apart later.
6. Hand-written de-duplication where the file already has a shorter way.

Two things I told them to leave alone. Both reviewers flagged them. Both are correct as they are.

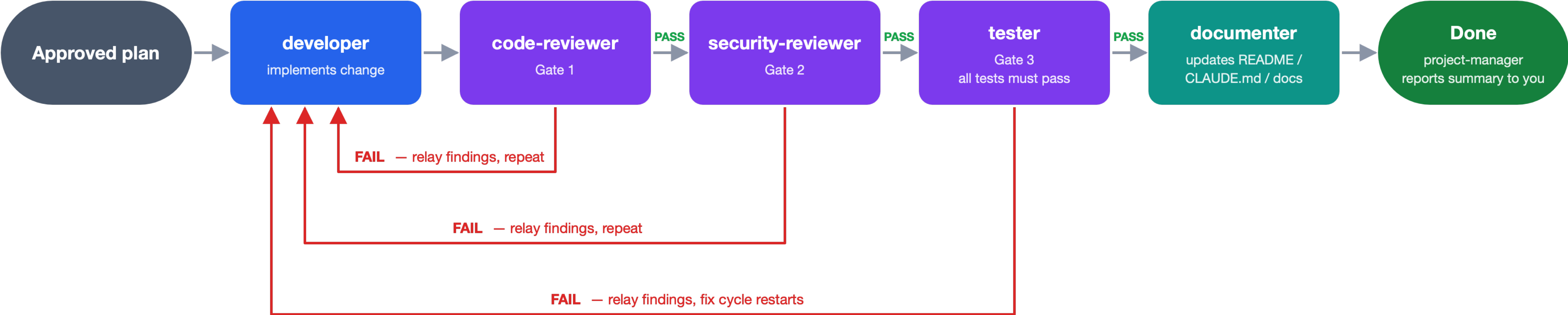
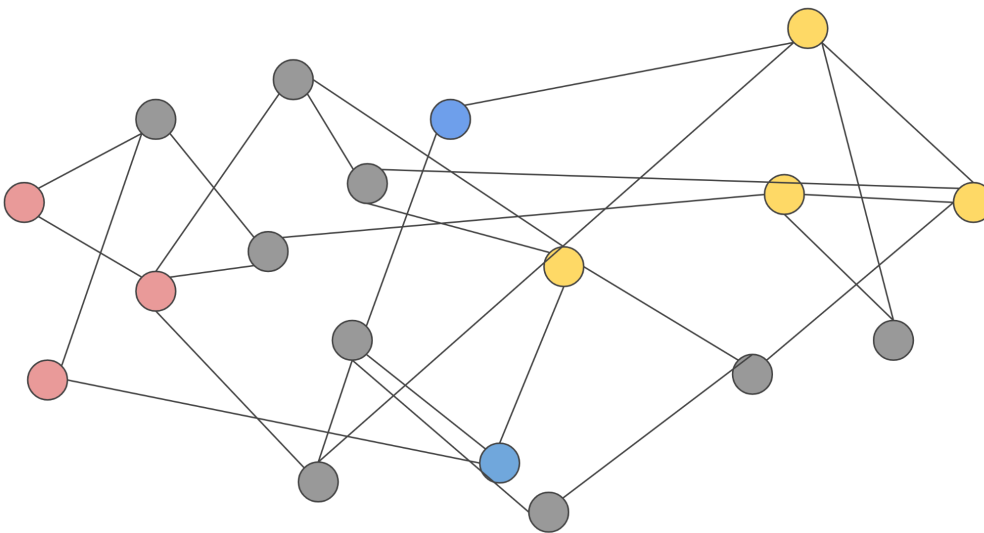
The PM is fixing the six now. Both review gates run again after.

Agents

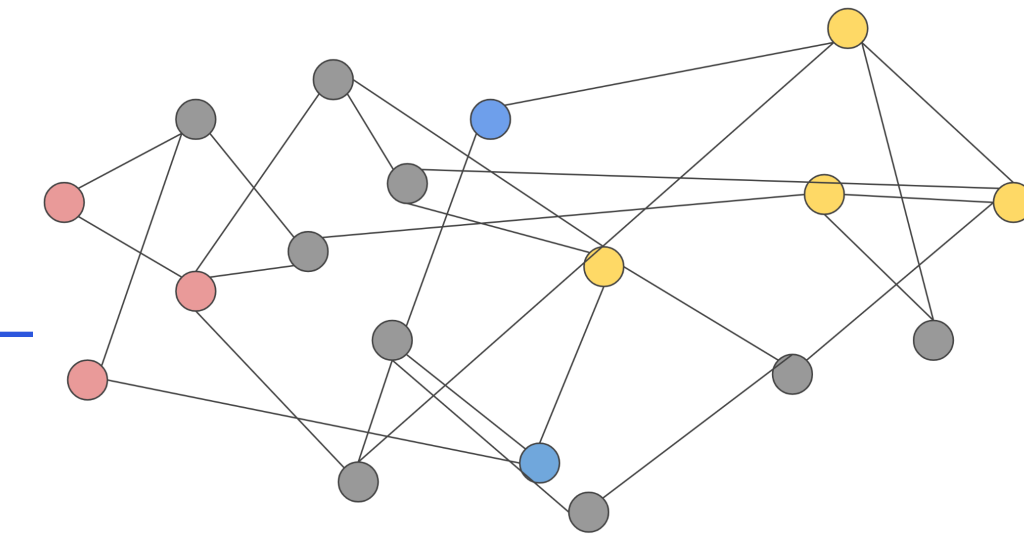


Agent	Role	Writes?	Commits?
project-manager	Coordinates the whole pipeline; delegates and routes	✗	✗
developer	Implements the change	✓ tests and code only	✗
code-reviewer	Reviews for correctness, clarity, and simplicity	✗	✗
security-reviewer	Reviews for OWASP Top 10 vulnerabilities	✗	✗
tester	Runs the test suite; commits on clean pass	✗	✓ after all gates pass
documenter	Updates README, CLAUDE.md, and design docs	✓ docs only	✗

Agent Pipeline



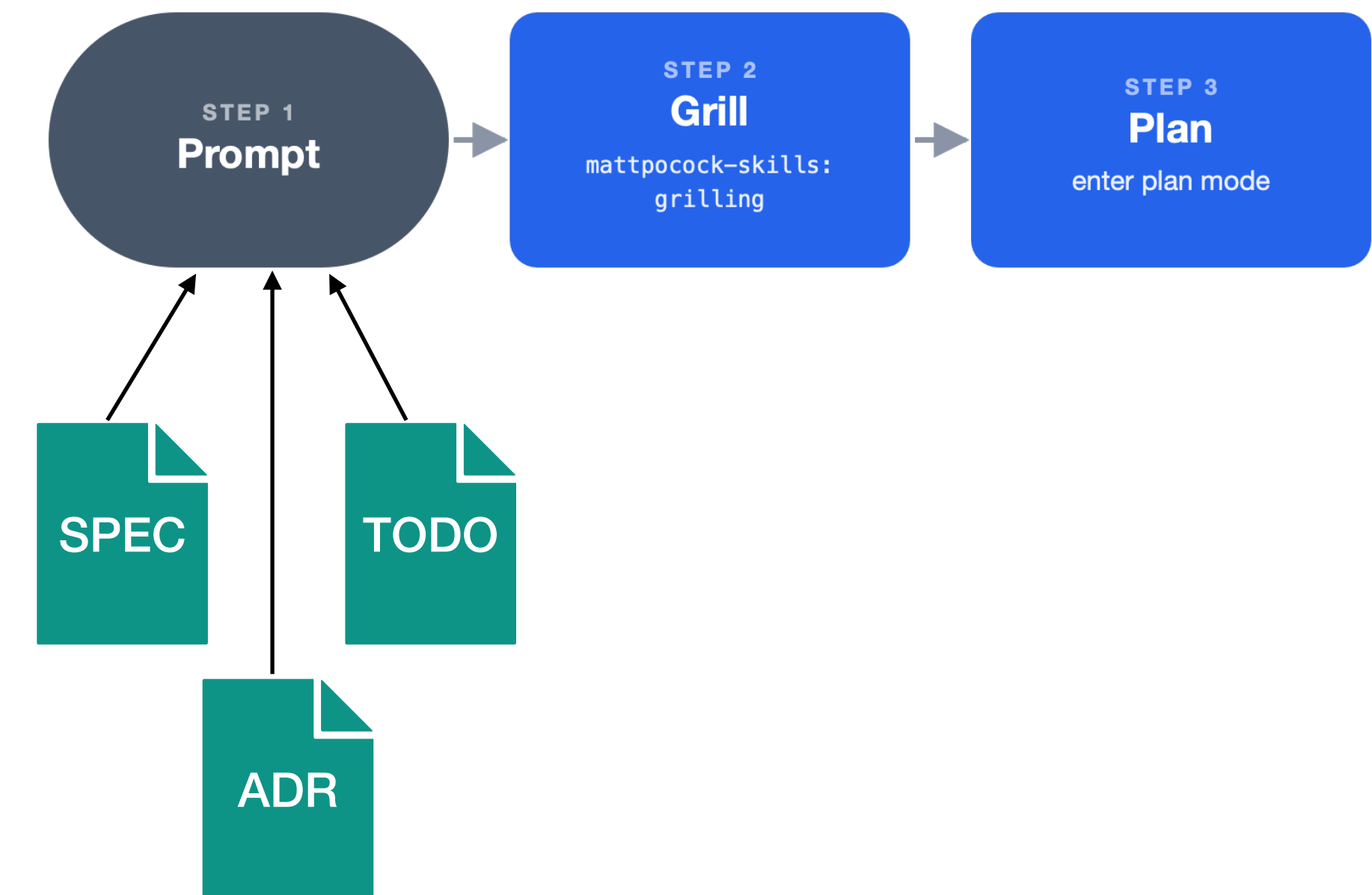
Getting to a Plan - Partial CLAUDE.md file



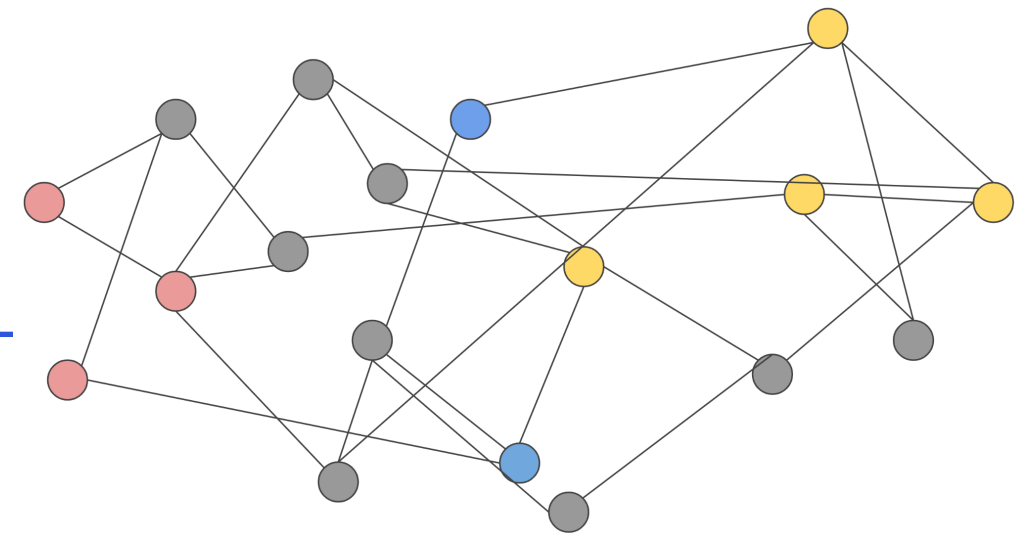
Feature workflow (mandatory)

Every request for a new feature MUST run this gate sequence, in order. Do not skip a gate unless the user explicitly waives it for that request.

1. ****Grill**** — invoke ``mattpocock-skills:grilling`` to stress-test the idea and requirements before any design work. If grilling surfaces new domain terms or an architectural decision, run ``mattpocock-skills:domain-modeling`` (CONTEXT.md / ADR) before planning. If a design question can only be answered by trying it, run ``mattpocock-skills:prototype``; if it needs external facts, ``mattpocock-skills:research``.
2. ****Plan**** — enter plan mode; get the plan approved before touching code.



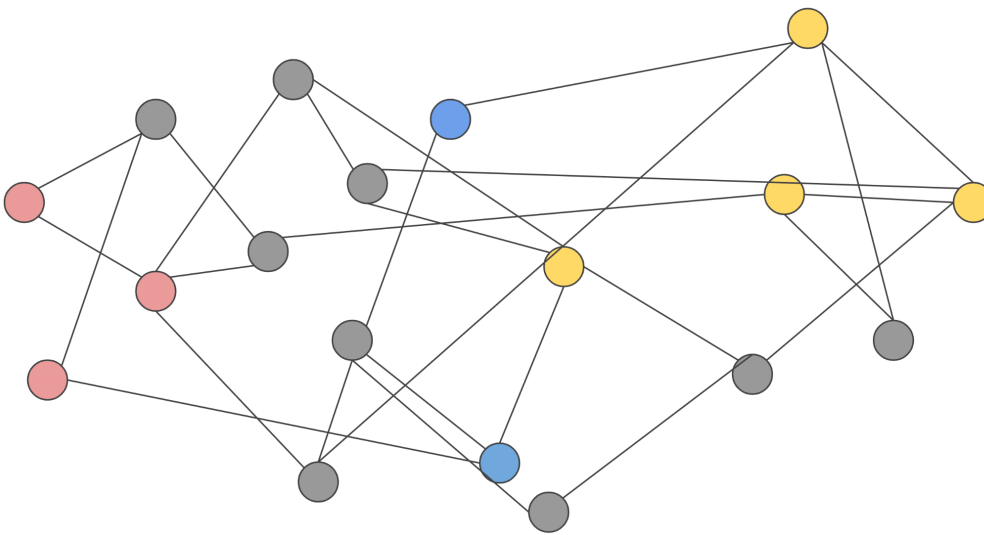
What if you don't have all those inputs?



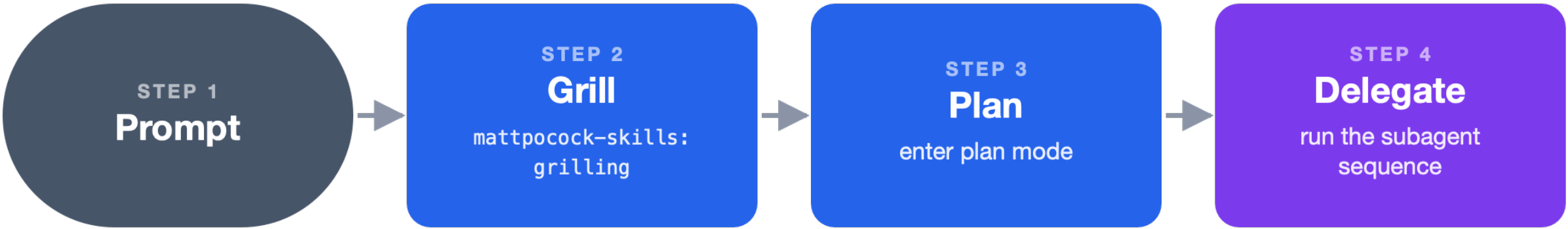
“I believe that failure is less frequently attributable to either insufficiency of means or impatience of labour than to a **confused understanding of the thing actually to be done.”**

–John Ruskin

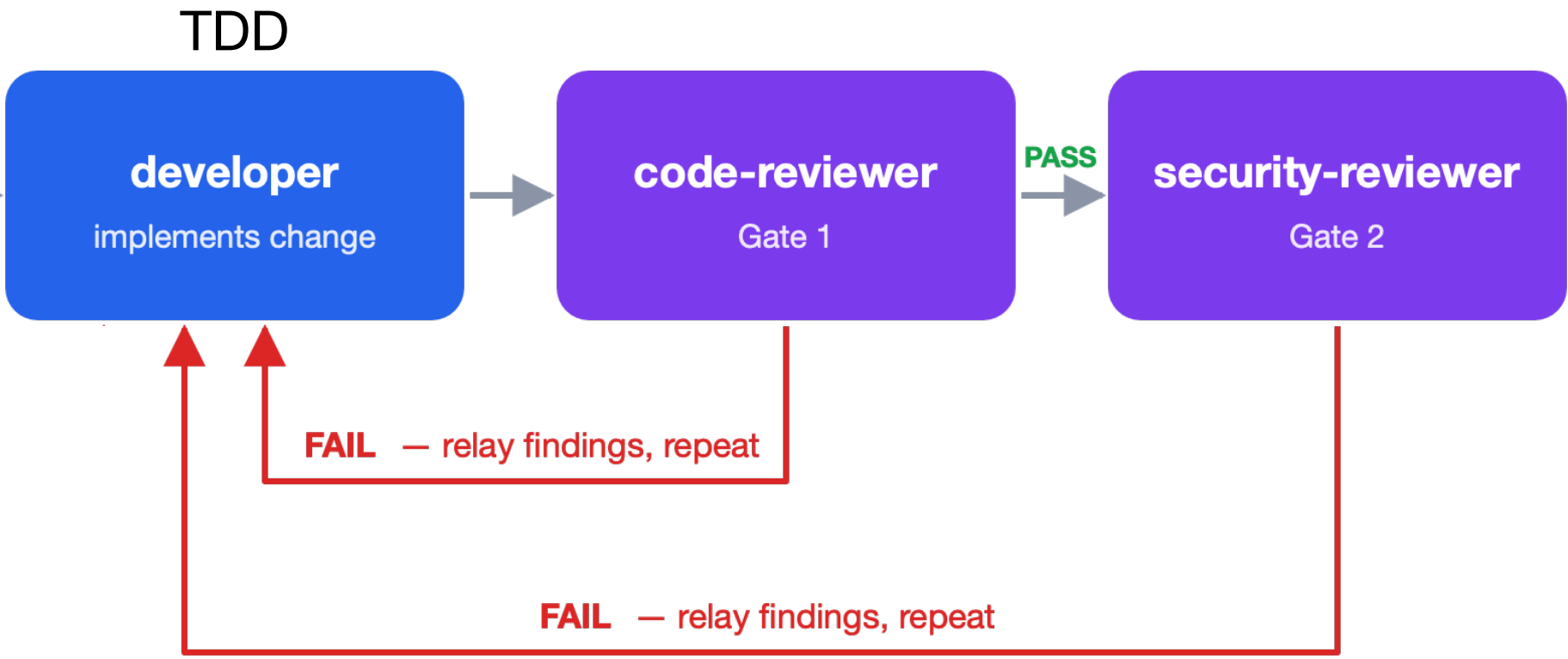
Implementation - Partial CLAUDE.md file



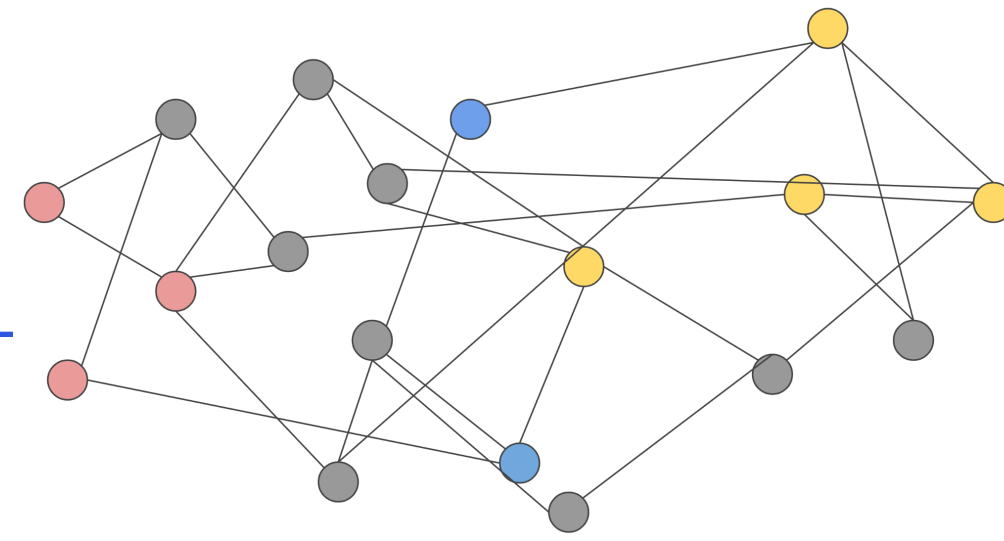
continued



3. **Delegate** — hand the approved plan to the `project-manager` agent. The PM runs the pipeline: `developer` implements via `mattpocock-skills:tdd` (test-first; red-green-refactor) → `code-reviewer` gate → `security-reviewer` gate; failed gates route fixes back to the `developer` until both PASS.

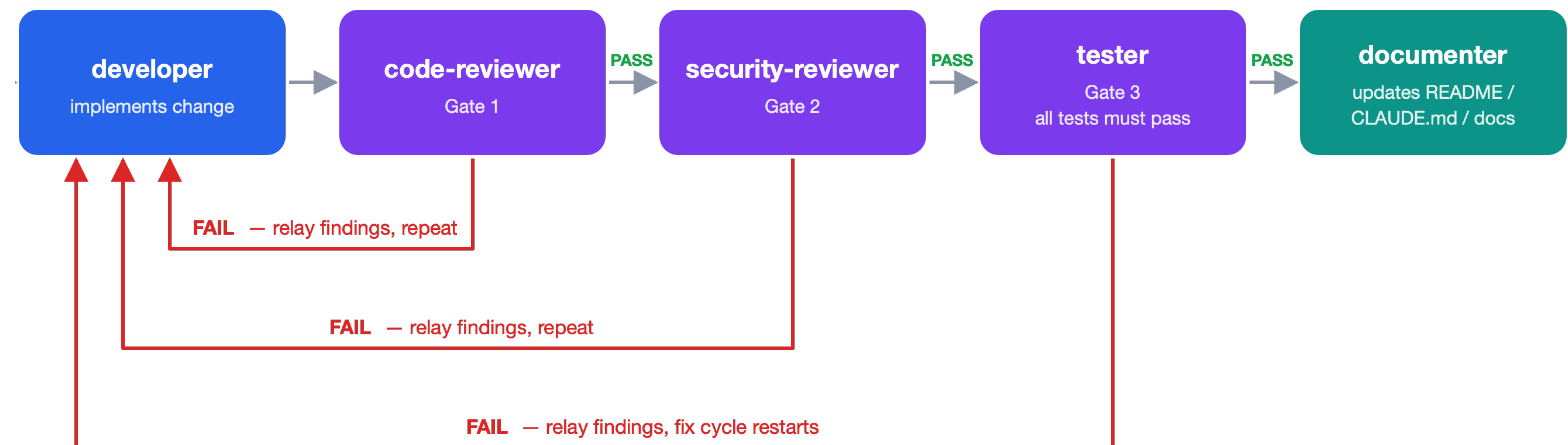


Review & test - Partial CLAUDE.md file

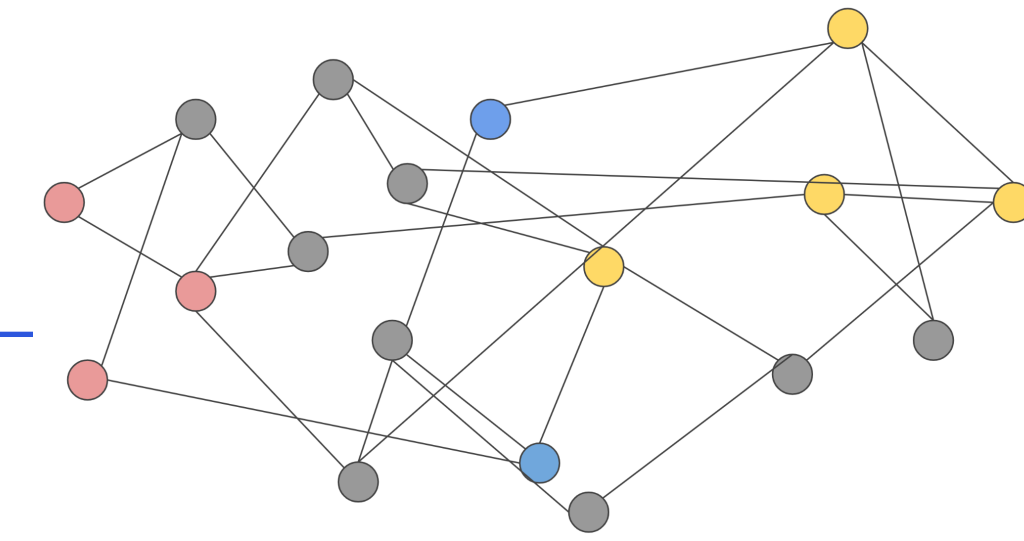


continued

4. ****Review**** — run ``mattpocock-skills:code-review`` against the merge-base (Standards axis + Spec axis vs the approved plan); route confirmed findings back through the ``project-manager``.
5. ****Commit**** — ``tester`` runs the full test suite(s) (both repos if the change touches both) and commits per repo only when everything passes; then ``documenter`` updates docs (including ``docs/integration-contract.md`` if the handoff changed).



Verify & Bug Workflow - Partial CLAUDE.md file



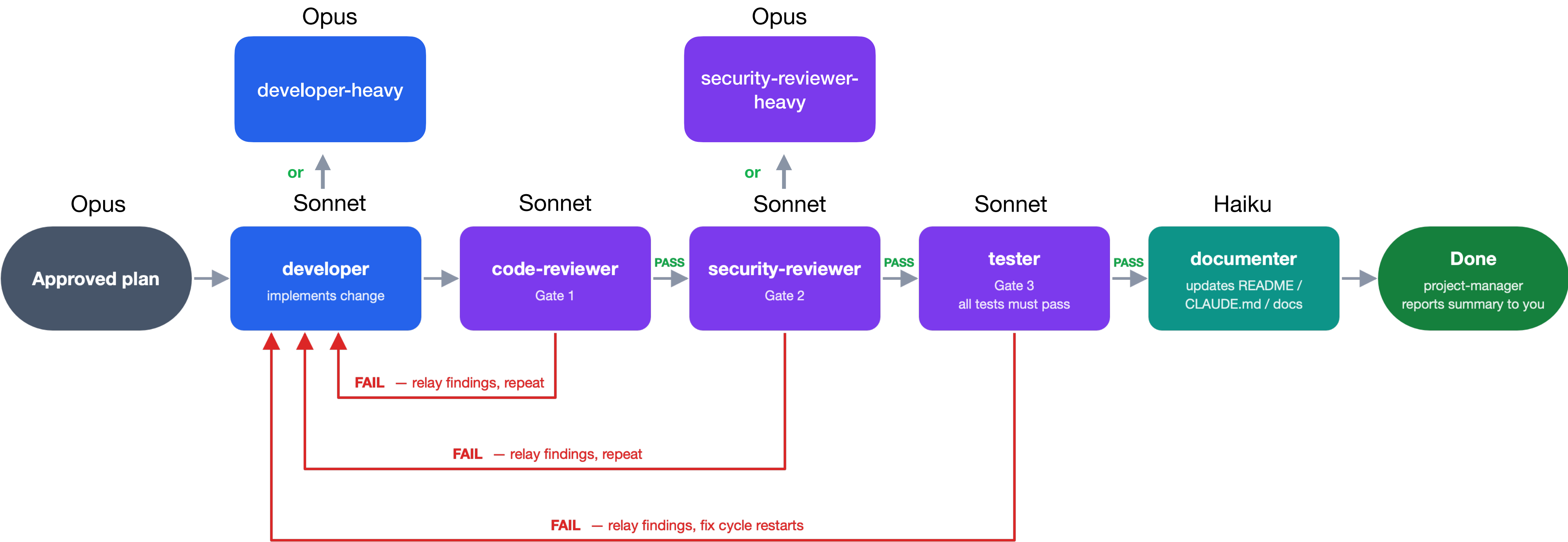
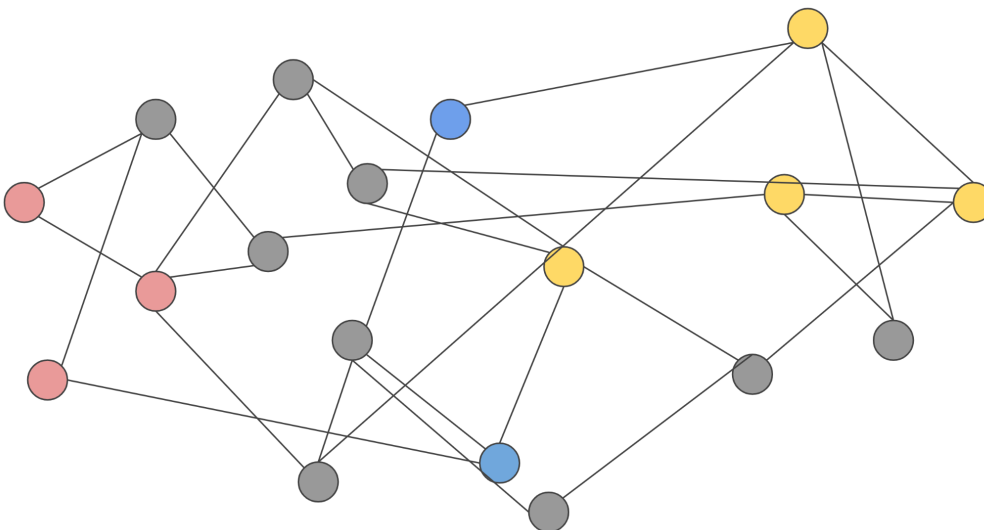
6. **Verify** — invoke ``superpowers:verification-before-completion``:
show fresh test output before claiming the feature is done.

Bug workflow

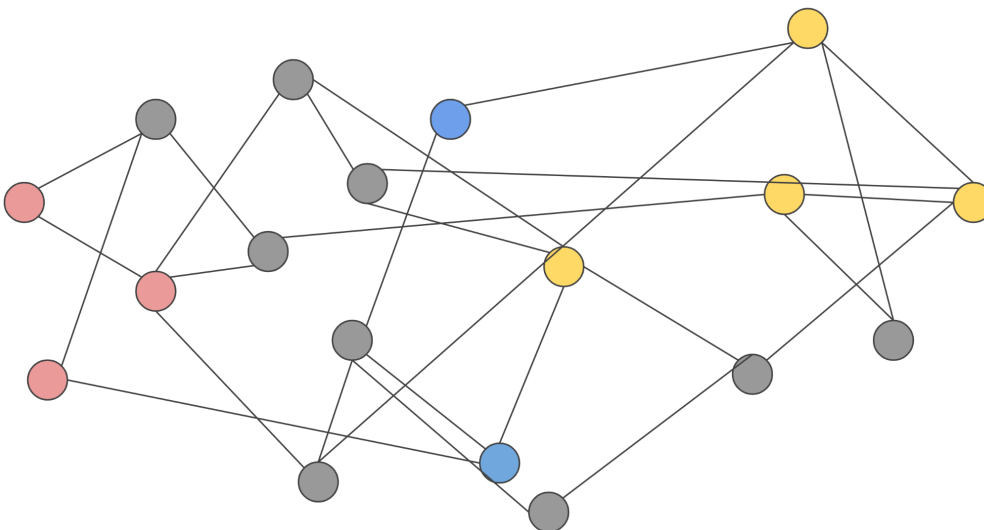
Bugs skip Grill/Plan. Diagnose first via

``mattpocock-skills:diagnosing-bugs`` (reproduce before you touch code), then enter the pipeline at **Delegate**: the ``developer`` turns the reproduction into a failing test, fixes it, and the same gates (code-reviewer → security-reviewer → tester → documenter) apply.

Agent Pipeline - Models



Catching and Resolving Hallucinations



Claude Opus 5.5
(the model itself)

One nit I already spotted: the help text now says "type a number in the Order box" twice. I will get that tightened.

security-reviewer
(Sonnet)

The security gate earned its money again

It failed three times. Every failure was a real defect. Two were **fail-open holes inside guards that claimed to fail closed**.

Mutation testing found worse. W4's certificate bytes had **zero** tests behind them – five deliberate breakages all went unnoticed. Now fixed and pinned.

One finding stands out. A test looked honest but proved nothing, because the test data was too simple: for a root, three different identifiers all collapse to the same value. So three wrong readings of the spec all passed. Fixed by adding data where the three values differ.

PM report and cycles
(Opus)

• **Review done. Two reviewers, working apart.**

Good news: the important stuff is right. All page links still work. Nothing broke. The tests pass.

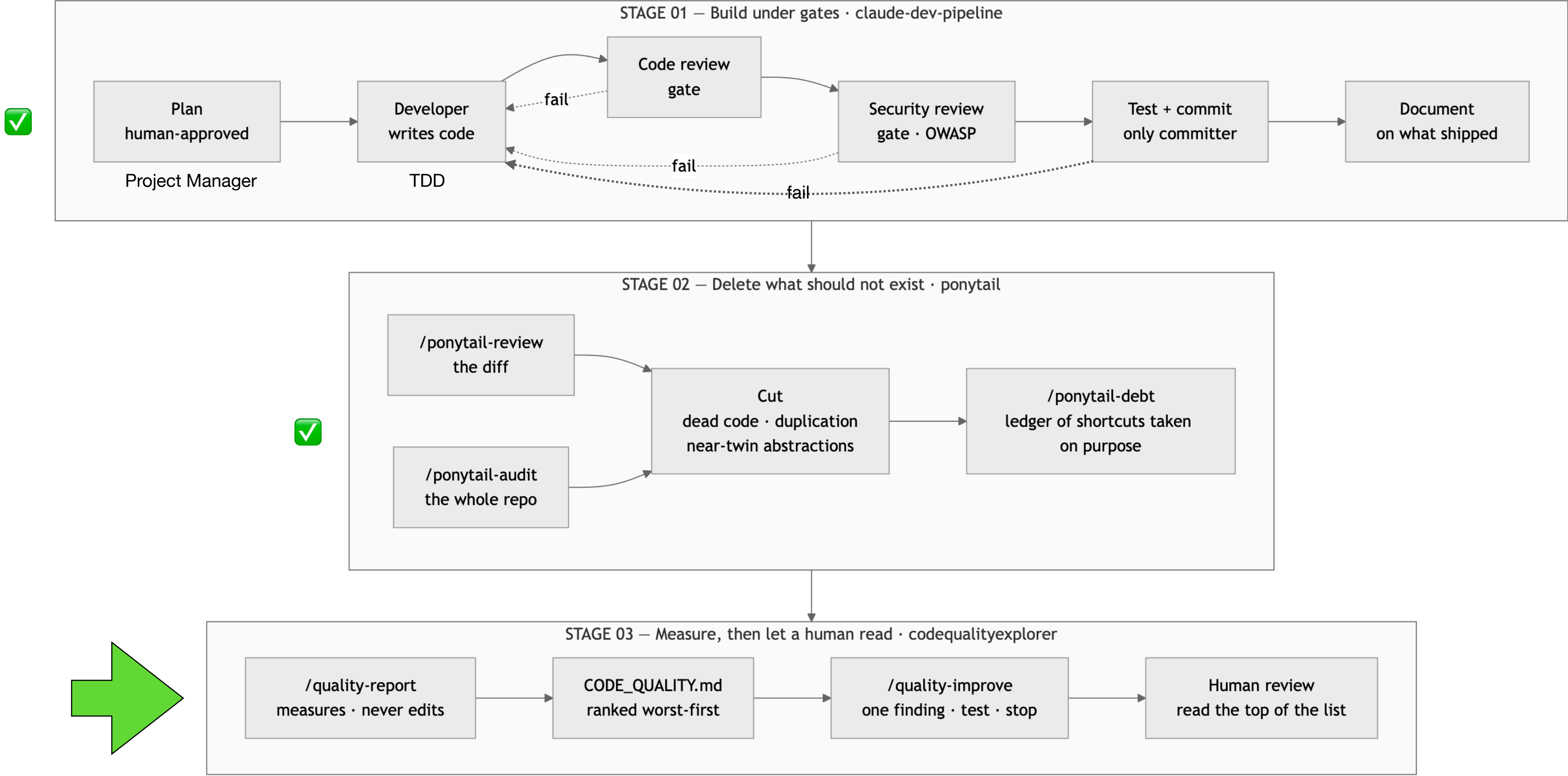
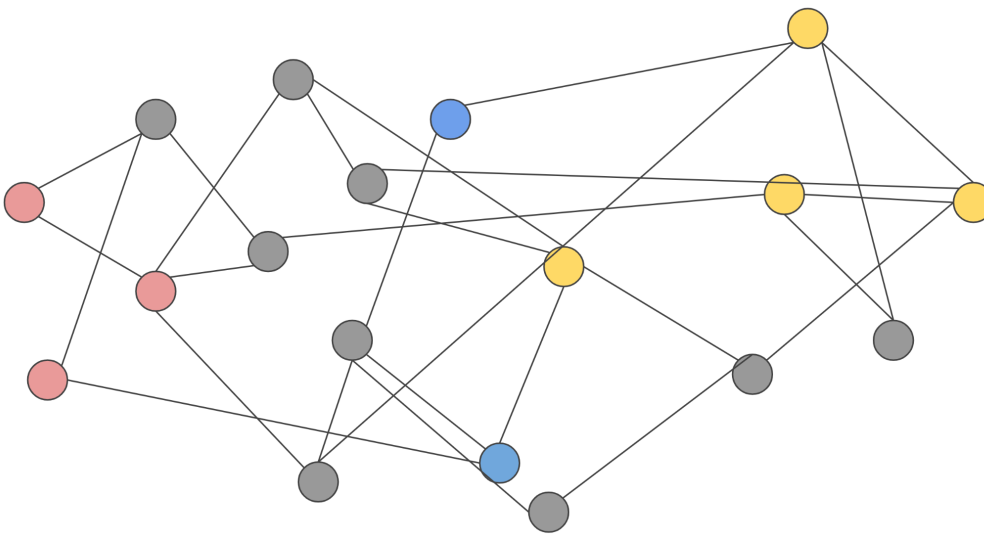
Six things to fix:

1. Single-source story link lists show at the wrong text size. Real bug you would see.
2. Some old CSS now does nothing. Delete it.
3. Trend line can end with "· updated " and no date.
4. It says "1 trends tracked". Bad grammar.
5. Ten lines of copy-pasted code. It will drift apart later.
6. Hand-written de-duplication where the file already has a shorter way.

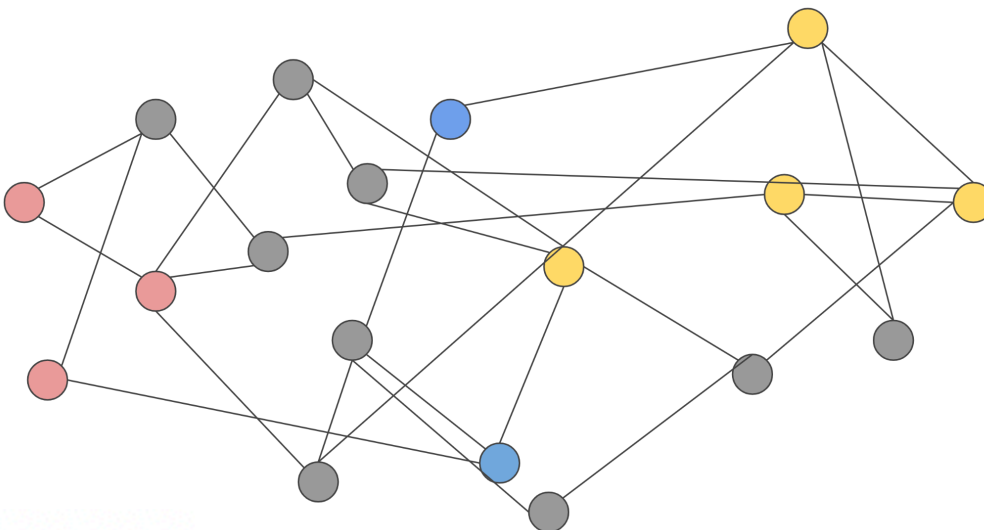
Two things I told them to leave alone. Both reviewers flagged them. Both are correct as they are.

The PM is fixing the six now. Both review gates run again after.

...and then



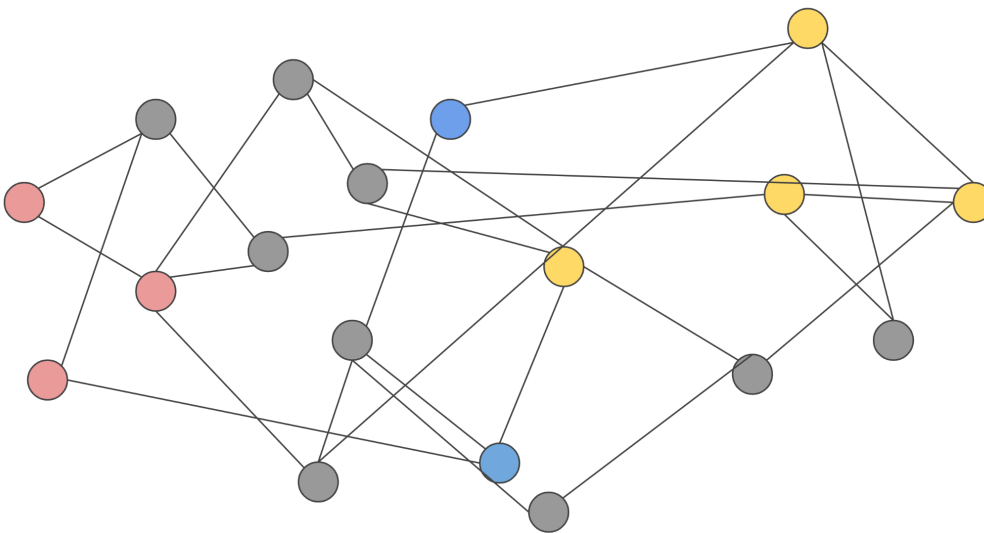
Code Quality Explorer - Report



Metrics

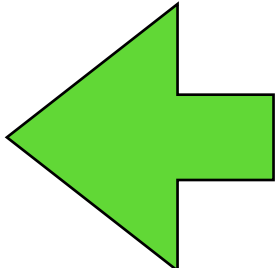
Metric	Threshold	Status	Compliance	Source
Cyclomatic complexity	> 10		1710 · 48 · 83	tool (radon)
Function length	> 50 lines		1964 · 25 · 56	tool (function_metrics)
Nesting depth	> 4 levels		2022 · 18 · 5	tool (function_metrics)
Argument count	> 5		1964 · 34 · 47	tool (function_metrics)
Code duplication	blocks ≥ 5 lines		15 regions	tool + judgment
Dead code	any		2	tool (ruff F401, vulture)
Error-handling smells	bare <code>except:</code>		none	tool (ruff E722)
File size	> 500 lines		13 · 0 · 12	tool (radon raw)
Comment quality	WHAT not HOW		205 · 38 (source only)	tool + judgment
Reusability	shared logic		2	judgment

Code Quality Explorer - Improve



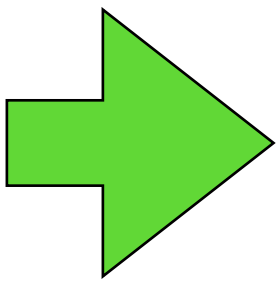
• **Q3 fixed.** The cleanest result yet.

	before	after	threshold	
complexity	45	1	10	
length	275	31	50	
nesting	3	0	4	
args	6	6	5	<- pre-existing, left alone



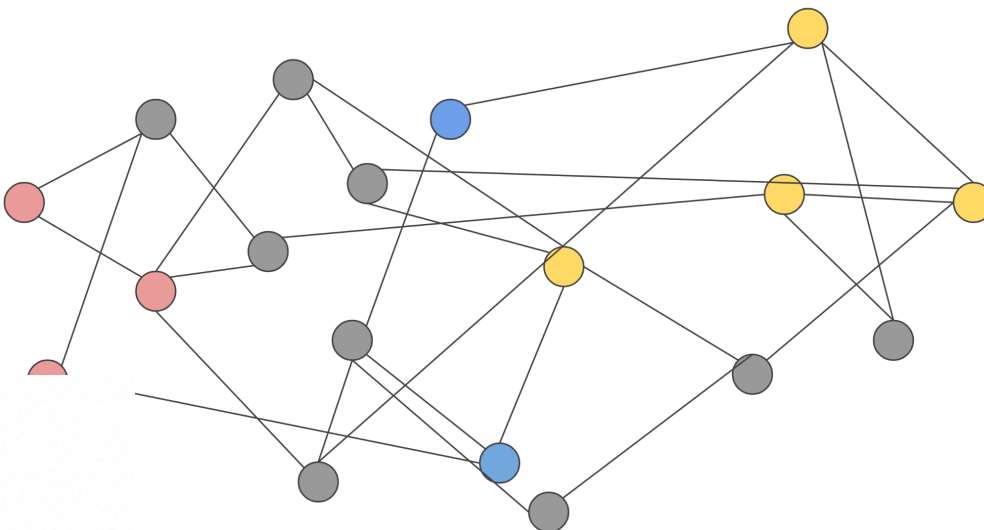
Metrics improved

All checks & tests green



Check	Result
StageTimer names and order	byte-identical – 10 timers, same order
kb_will_commit	same line, step-8 block untouched
--load-items trend guard	intact, replay still makes zero trend calls
Suite	2240 passed – identical to baseline

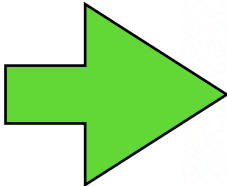
Code Quality Explorer - Report (After)



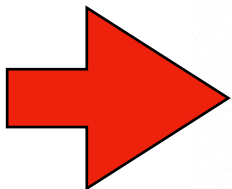
Metrics

Metric	Threshold	Status	Compliance	Source
Cyclomatic complexity	> 10	● 96.6% pass	● 2042 · ● 21 · ● 73	tool (radon)
Function length	> 50 lines	● 98.1% pass	● 2209 · ● 26 · ● 43	tool (function_metrics)
Nesting depth	> 4 levels	● 100% pass	● 2262 · ● 16 · ● 0	tool (function_metrics)
Argument count	> 5	● 99.3% pass	● 2204 · ● 59 · ● 15	tool (function_metrics)
Code duplication	blocks ≥ 5 lines	● 5.9% (estimated)	● 375 groups, 90% of them test fixtures	estimated
Dead code	any	● 99.6% (491/493)	● 2 reached only by their own tests	tool (vulture, ruff F401)
Error-handling smells	bare except :	● 0.0 / 1k lines	● none	tool (ruff E722)
File size	> 500 lines	● worst 11.5x limit	● 25 · ● 2 · ● 15	tool
Comment quality	WHAT not HOW	● 99.7% pass (source)	● 340 · ● 1 of 341 source functions > 10 lines	tool + judgment
Reusability	shared logic	● 5.9% (estimated)	● 375 groups, 38 of them source-only	judgment

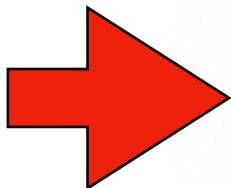
Still work
to do



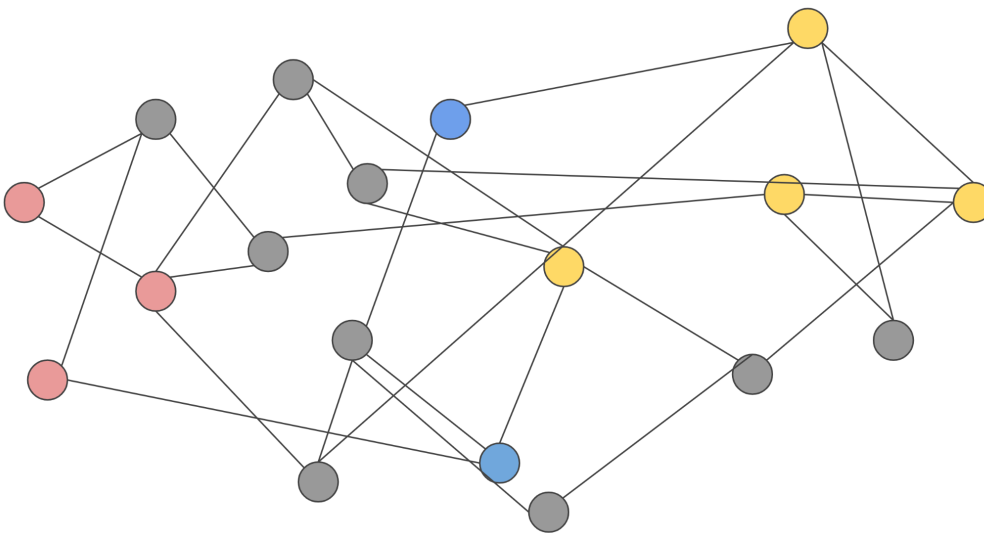
Needs fixing



Test files

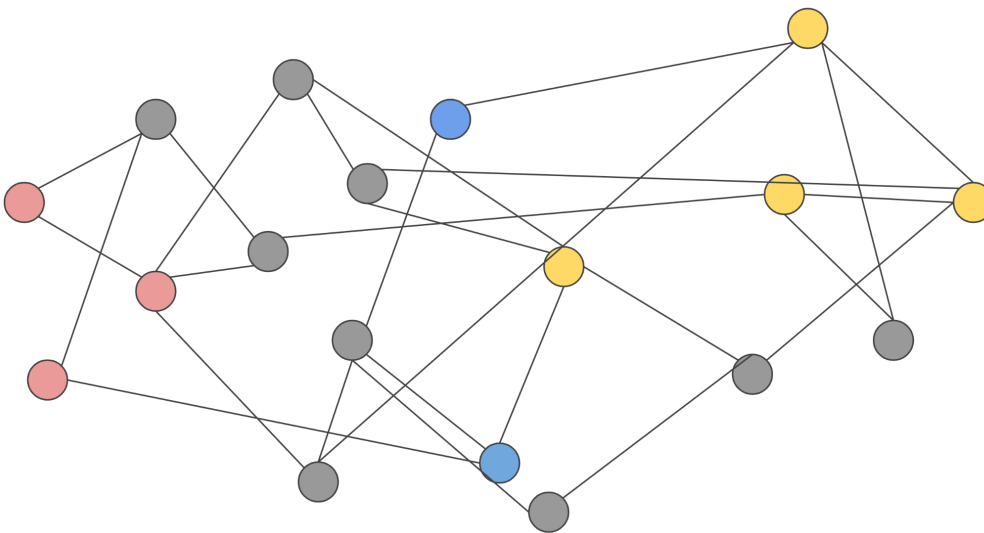


Sources



Project	URL
Claude Dev Pipeline	https://github.com/prototipo/claude-dev-pipeline
Matt Pocock's skills	https://github.com/mattpocock/skills
Ponytail	https://github.com/DietrichGebert/ponytail
Code Quality Explorer	https://github.com/prototipo/codequalityexplorer

Where next?

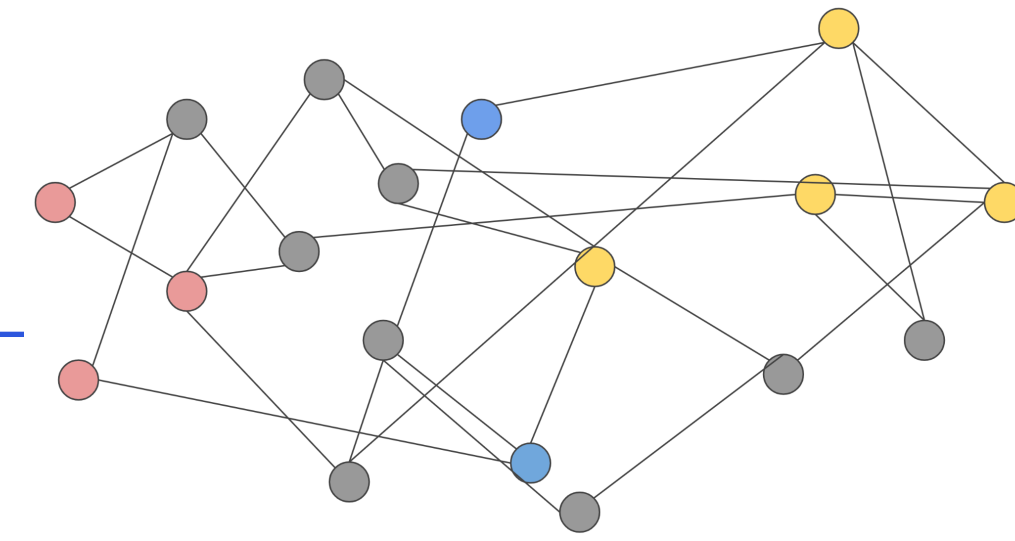


Maybe integrate Jev or Laya, general-purpose classifiers.

Question type	Goal	Returns
Choice	Choose an option from a list	choice, probabilities, confidence
Score	Score the state on a rubric	score, probabilities, confidence
Noul	Is this statement true?	noul (0–1)

Jev: <https://typesafe.ai>
Laya <https://github.com/NandhaKishorM/laya>

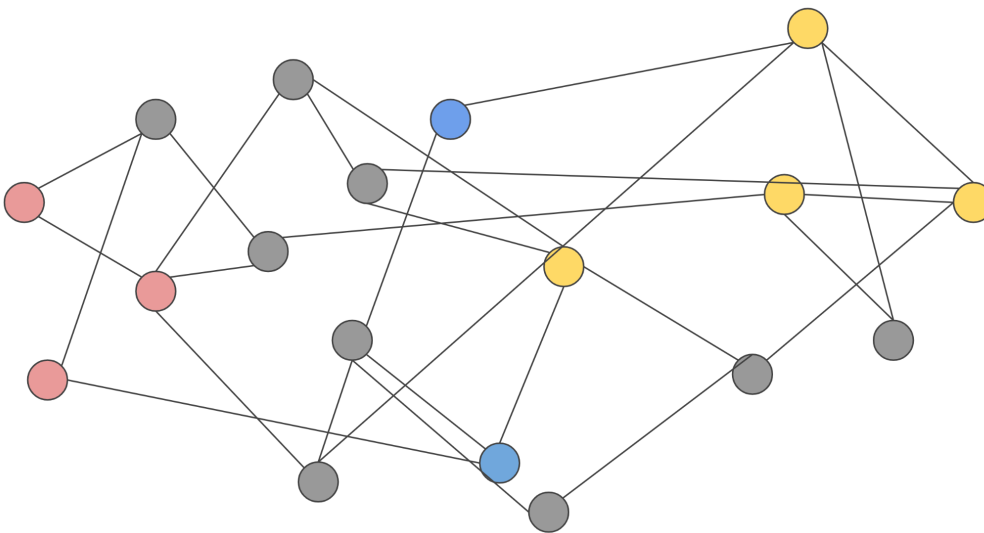
Where next?



Possible classifier use cases:

- Intake routing, to replace decisions currently falling to the Opus project-manager or to CLAUDE.md prose
- Gate triage, e.g., splitting the security question into atomic Nouns over the diff
- Detecting a fix loop that isn't converging
- Deciding whether the documenter needs to run
- Spec-conformance pre-checks
- Checking comment quality

Reconsider TDD inside the agent loop



Costs and benefits
when an agent
does TDD by itself

Cost: At least 3x the tokens

Cost: Prompt and eval investment to get agents to reliably follow TDD

Surface what you ain't gonna need (YAGNI)

Lost with human out of the loop

Confidence and learning

Lost with human out of the loop

Fewer tautological tests

Happened during the experiment in spite of TDD

Less complex tests

Opus evaluator didn't find a meaningful difference

More effective tests (regression)

Solutions didn't show meaningful difference in mutation scores

Better design

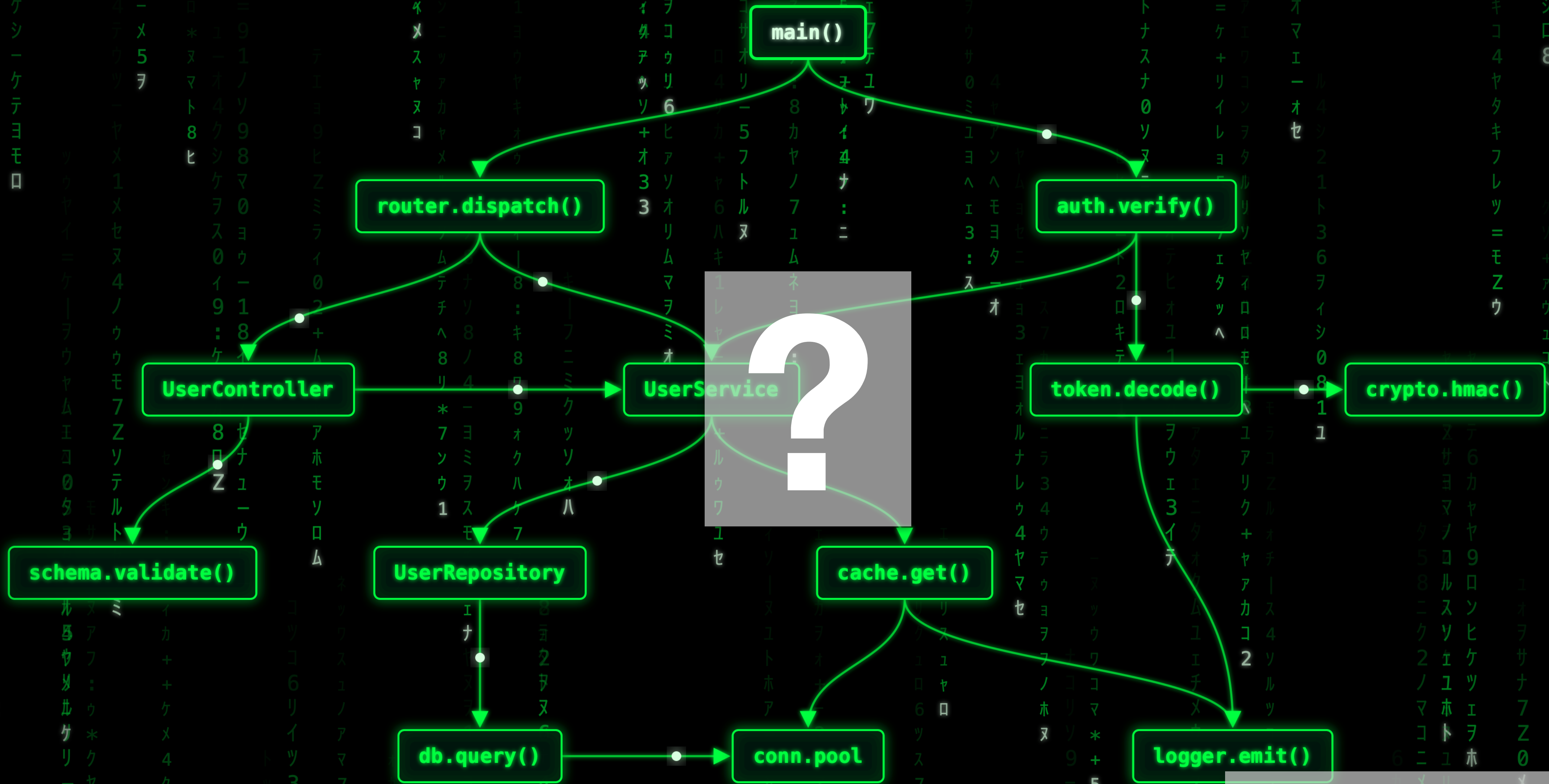
Opus evaluator posed **hypothesis that agents are better at up front design** than TDD step by step

Fast feedback / easier to debug

Experiment tasks not suitable to give any data about this

→ Experiment didn't surface any data that would reduce my skepticism about the usefulness of TDD in the loop - no proof of the opposite either, a proper eval would be needed.

Birgitta Boeckeler | August 2026 | <https://martinfowler.com/articles/exploring-gen-ai/tdd-in-the-agent-loop.html>



<https://hyland-wood.org>